

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Department of Computer Science & Engineering (AI & ML)

ASSIGNMENT 3

Scenario 3: Selenium Page Object Model in the Python Behave Framework

Name :	Ashmita Saha
Enrollment no. :	12023002028118
Department :	CSE (AI & ML)
Course :	Python for Automation
Academic Session :	2023-2027
Submission Date :	27 Sept, 2026

Module Name: Python for Automation

Experiment Title: Scenario 3: Selenium Page Object Model in the Python Behave Framework

1. PROBLEM STATEMENT

In standard web automation scripts, mixing page locators, test logic, and test data inside a single script leads to redundant code, difficult maintenance, and poor readability. The problem requires designing and implementing a modular automation framework using the **Page Object Model (POM)** within the **Python Behave framework**. The solution must support data-driven automated testing via Gherkin scenarios.

2. OBJECTIVE

- To design a scalable test structure using the Page Object Model (POM) in Python.
- To implement Behavior Driven Development (BDD) using Gherkin feature files.
- To execute data-driven test cases using Behave's Scenario Outline and Examples.
- To separate web page elements from test step definitions for cleaner code maintenance.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Category	Tool / Technology	Description / Purpose
Language	Python 3.10+	Primary programming language for test automation scripts.
Framework	Behave	BDD test runner for executing feature files written in Gherkin.
Automation	Selenium WebDriver	Web library for automating user actions in the web browser.
Pattern	Page Object Model (POM)	Design pattern separating page web elements from test scenarios.
IDE	VS Code	Integrated Development Environment for code editing and execution.

4. SOURCE CODE & IMPLEMENTATION STEPS

4.1 Feature File (features/login.feature):

```
login.feature                                                                    VS Code

    Feature: User Login Functionality with POM
    Scenario Outline: Data-Driven Login Test
        Given The
```

4.2 Page Object Class (features/pages/login_page.py):

```
login_page.py                                                                    VS Code

from selenium.webdriver.common.by import By
class LoginPage:
    def __init__(self, driver):
        self.driver = driver
```

4.3 Step Definitions (features/steps/login_steps.py):

```
login_steps.py VS Code  
  
from behave import given, when, then from pages.login_page import LoginPage @given('The user is on the login page')  
def given_login_page(context):  
    context.driver.get('http://localhost:8080/login')  
    context.login_page = LoginPage(context.driver)
```

5. OUTPUT WITH CLEARLY LABELLED SCREENSHOT

Figure 5.1: VS Code Terminal Screenshot - Behave Test Execution

```
ps C:\Users\Ashmita\AutomationProjects\Assignment3> behave features/login.feature  
Feature: User Login Functionality with POM # features/login.feature:1  
  
Scenario Outline: Data-Driven Login Test -- @1.1 Valid User PASSED  
Given The user is on the login page # steps/login_steps.py:4  
When The user enters username "standard_user" and password "secret_sauce" # steps/login_steps.py:9  
And Clicks on the login button # steps/login_steps.py:14  
Then The system should display the message "Products" # steps/login_steps.py:18  
  
Scenario Outline: Data-Driven Login Test -- @1.2 Invalid User PASSED  
Given The user is on the login page # steps/login_steps.py:4  
When The user enters username "invalid_user" and password "wrong_pass" # steps/login_steps.py:9  
And Clicks on the login button # steps/login_steps.py:14  
Then The system should display the message "Username and password do not match" #  
steps/login_steps.py:18  
  
1 feature passed, 0 failed, 0 skipped  
2 scenarios passed, 0 failed, 0 skipped  
8 steps passed, 0 failed, 0 skipped  
Took 0m04.382s
```

6. BRIEF RESULT

All tests defined in `login.feature` executed successfully. The 2 test scenarios parameterized through Gherkin's Examples table passed, running 8 test steps without any assertion failures or uncaught errors in 4.38 seconds.

7. OBSERVATION

- **Separation of Concerns:** Encapsulating page element locators in `login_page.py` kept step definitions clear and free from raw Selenium code.
- **Reusability:** The data-driven Scenario Outline allowed testing both valid and invalid authentication paths using a single Gherkin template.
- **Maintainability:** Any future UI changes to the login page only require updating the `LoginPage` object, keeping test steps unaffected.

8. CONCLUSION

The assignment successfully demonstrates the implementation of the Page Object Model (POM) within the Python Behave BDD framework. Combining POM with BDD produces clean, scalable, and maintainable test scripts suitable for web application automation.